

Capítulo 2: Recursividad

Laura Viviana Galvis

Escuela de Ingeniería de Sistemas e Informática
Facultad de Ingenierías Fisicomecánicas
Universidad Industrial de Santander

2025-2

Escuela de
Ingeniería de
Sistemas e
Informática

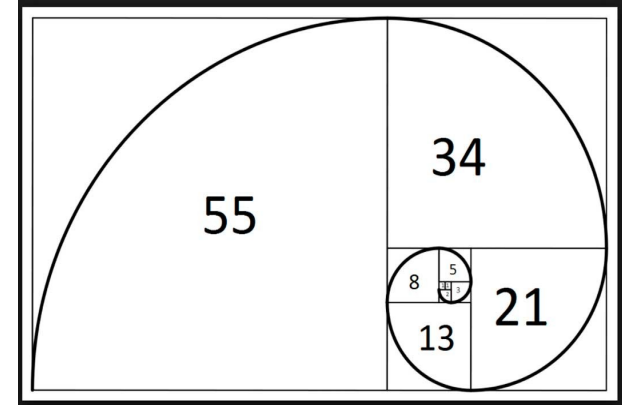
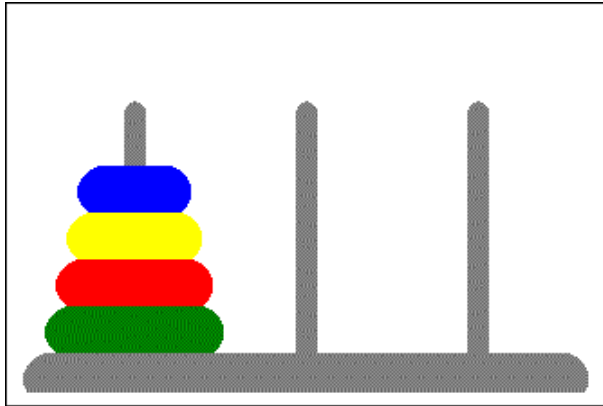


Universidad
Industrial de
Santander



Contenidos del capítulo

1. Definición y métodos recursivos
2. Factorial de un número mediante recursividad
3. Sucesión de Fibonacci
4. Torres de Hanoi



1. Definición y métodos recursivos

¿Qué es la recursividad?

- Una recurrencia = una recursión = algo que se llama a sí mismo
- Procedimiento que permite resolver un problema complejo reduciéndolo en uno o más subproblemas

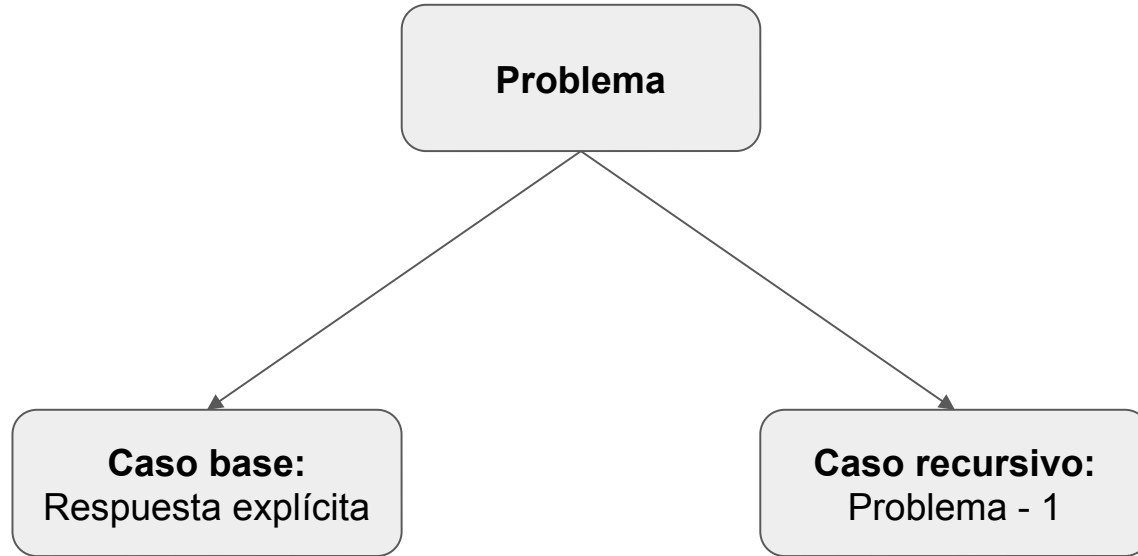


Características:

- Misma estructura que el problema original
- Más simple que el problema original
 - **i.e. un escalón a la vez**
- Cada subproblema, se divide en otros subproblemas cada vez más simples
- Los subproblemas llegan a ser tan simples, que no necesitan ser divididos nuevamente



Estructura de un algoritmo recursivo



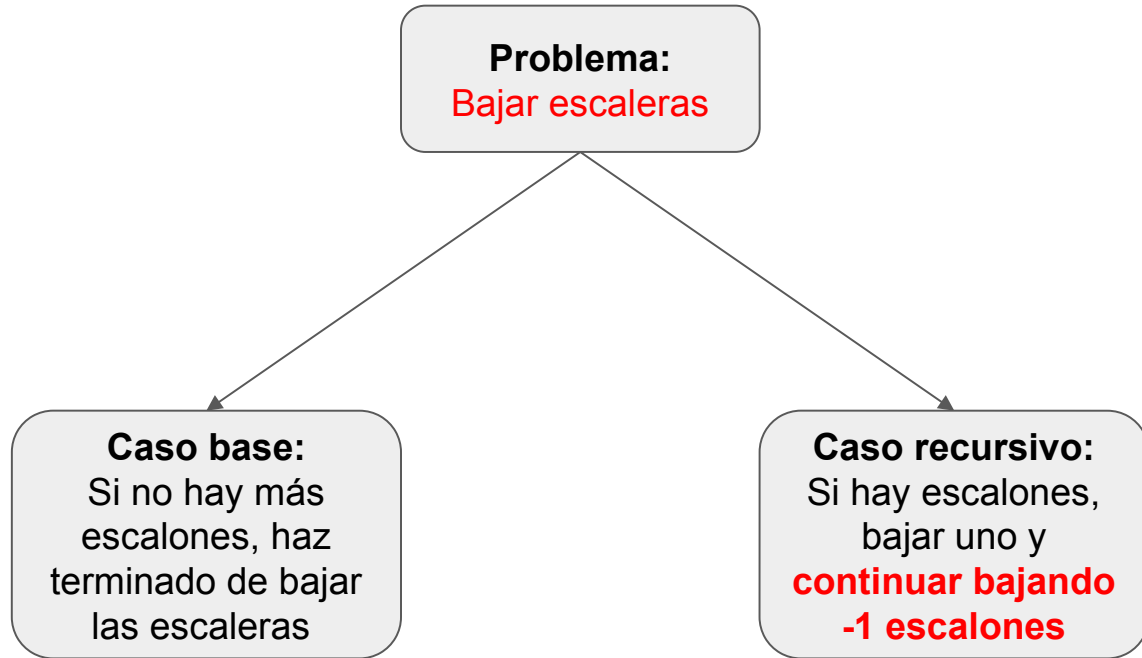
Todo algoritmo recursivo necesita un “**Caso base**”, para determinar el **final** de la recursión.

Ejemplo: Bajar/subir escaleras tiene un fin; o el factorial/fibonacci no está definido para negativos.

Ejemplo: Bajando escaleras



Adobe Stock | #130664387



¿Cómo sería el pseudocódigo?

Ejemplo: Bajando escaleras



Adobe Stock | #130664387

```
1  public class Bajando_Escalera {  
2  
3      public static void main(String[] args){  
4  
5  
6      }  
7  
8      public void bajarEscalera (int numeroEscalones){  
9          if ( ) { \\Caso base  
10  
11      }  
12      else { \\Caso recursivo  
13  
14  
15  
16      }  
17  }  
18 }
```

Ejemplo: Bajando escaleras



Adobe Stock | #130664387

```

1  public class Bajando_Escalera {
2
3      public static void main(String[] args){
4          Bajando_Escalera se = new Bajando_Escalera();
5          se.bajarEscalera(10);
6      }
7
8      public void bajarEscalera (int numeroEscalones){
9          if (numeroEscalones == 0){ \\Caso base
10             System.out.println("Has terminado de bajar las escaleras");
11         }
12         else{ \\Caso recursivo
13             System.out.println("Bajando escalon " + numeroEscalones +
14                 ". Ahora faltan " + (numeroEscalones-1));
15             bajarEscalera(numeroEscalones-1);
16         }
17     }
18 }
    
```

2. Factorial de un número mediante recursividad

Definición de Factorial

- El factorial de un entero positivo n , se define como el producto de todos los números **enteros positivos** desde **1** hasta n .

Por ejemplo:

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 5 \times 4! = 120$$

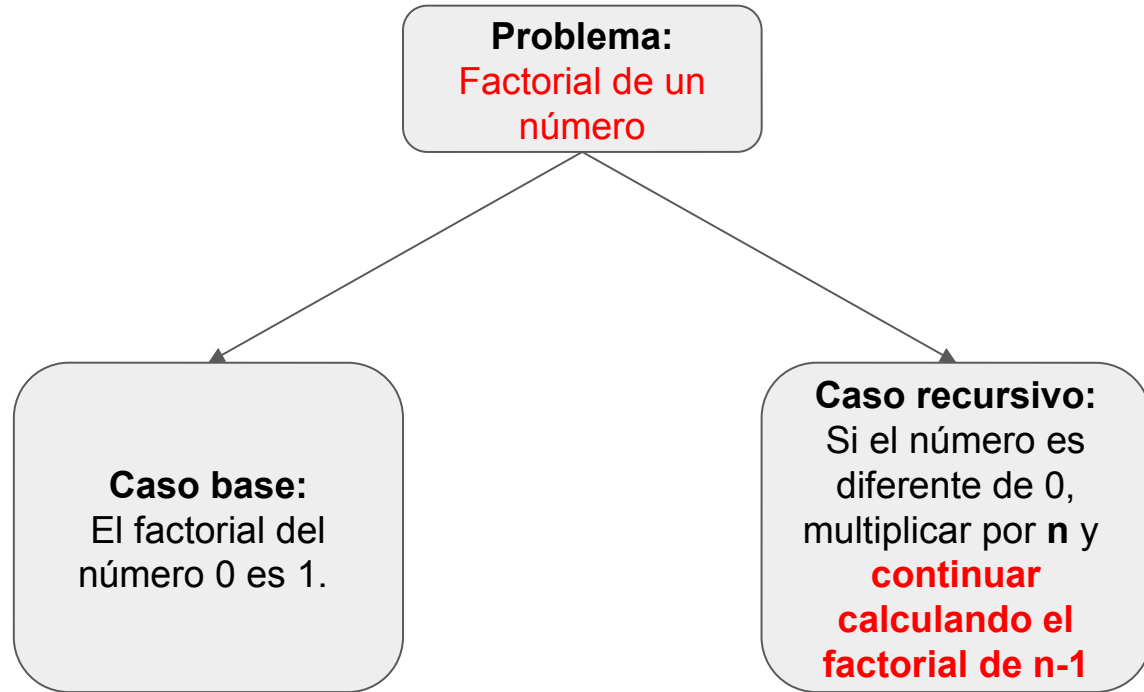
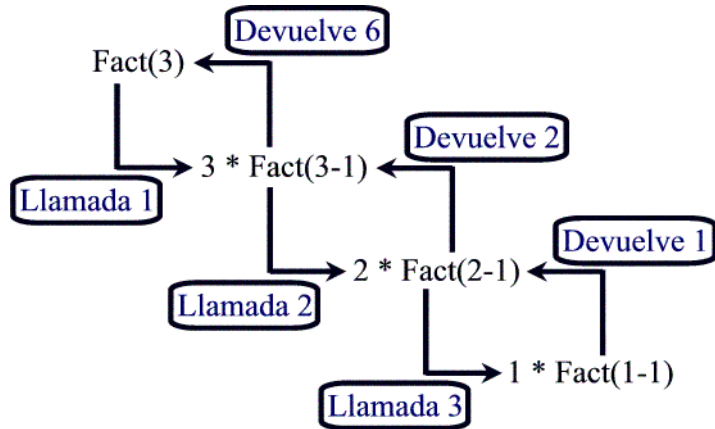
- El factorial de n representa el número de formas distintas de ordenar n objetos distintos (elementos sin repetición).

$$n! = 1 \times 2 \times 3 \times 4 \times \dots \times (n-1) \times n$$

$$n! = \prod_{k=1}^n k$$

$$n! = \begin{cases} 1 & \text{si, } n < 2 \\ (n-1)! \times n & \text{si, } n > 1 \end{cases}$$

Estructura recursiva del factorial de un número



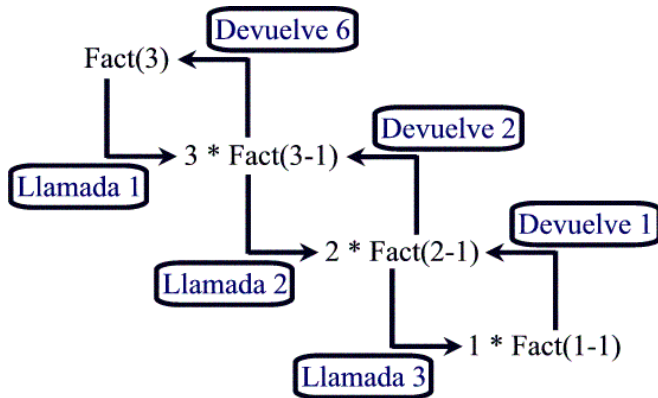
¿Cómo sería el pseudocódigo?

Factorial en Java (Recursivo)

```

1  public static int factorial(int n) {
2      if (n == 0) { \\Caso base
3          return 1;
4      } else { \\Caso recursivo
5          return n * factorial(n - 1);
6      }
7  }

```



```

1  import java.util.Scanner;
2
3  public class Recursividad_Factorial {
4
5      public static void main(String[] args) {
6
7          Scanner sc = new Scanner(System.in);
8          System.out.println("Digite un numero para calcular su factorial: ");
9
10         int num = sc.nextInt();
11         System.out.println("El factorial de " + num + " es = " + factorial(num));
12     }
13
14     public static int factorial(int n) {
15         if (n == 0) { \\Caso base
16             return 1;
17         } else { \\Caso recursivo
18             return n * factorial(n - 1);
19         }
20     }
21 }

```

Factorial en Java (con ciclos)

```
1  public static int factorialCicloWhile(int n){
2      int factor = 1;
3      while (n != 0) {
4          factor = factor * n;
5          n = n - 1;
6      }
7      return factor;
8  }
```

While-loop

```
1  public static int factorialCicloFor(int n) {
2      int factor = 1;
3      for (int i = 1; i<=n; i++) {
4          factor = factor * i;
5      }
6      return factor;
7  }
```

For-loop

```
1  public static int factorial(int n) {
2      if (n == 0) {
3          return 1;
4      } else {
5          return n * factorial(n - 1);
6      }
7  }
```

Rekursivo

Ventajas / desventajas de recursividad:

- La recursividad permite una mejor legibilidad.
- La recursividad puede ser más costosa debido al múltiple llamado de funciones.
- Puede desencadenar un desbordamiento de memoria
- En el caso de Fibonacci, se llaman múltiples veces los mismos casos (costo injustificado)

3. Sucesión de Fibonacci

Sucesión de Fibonacci

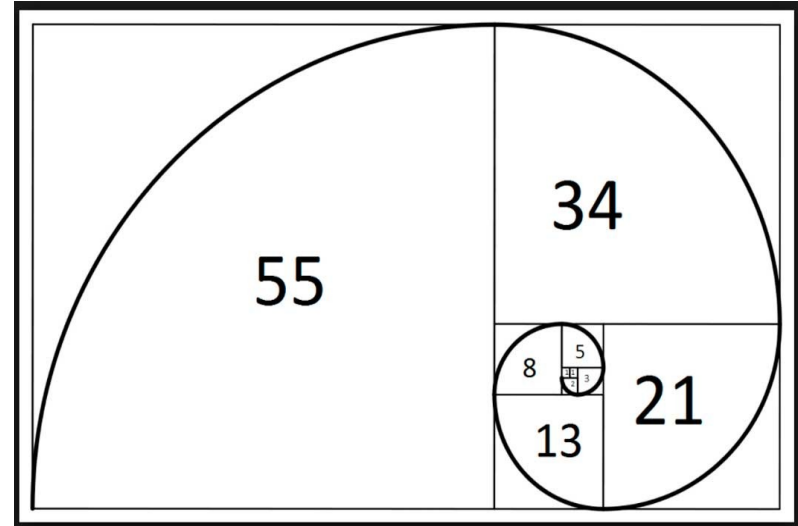
- Es una sucesión infinita de números naturales.
- La sucesión comienza con los números 0 y 1;
- A partir de estos, cada término siguiente es la **suma de los dos anteriores**.

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2}$$

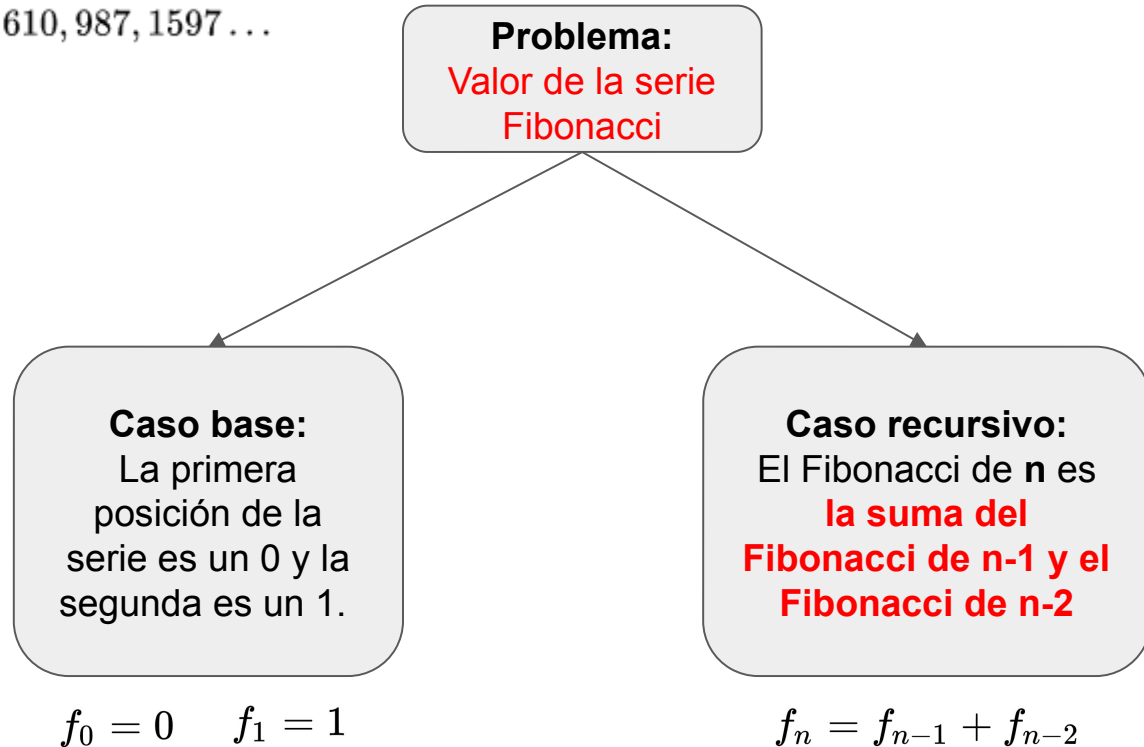
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597 ...



Estructura recursiva de la serie Fibonacci

0,1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597 ...

¿Cómo sería el pseudocódigo?



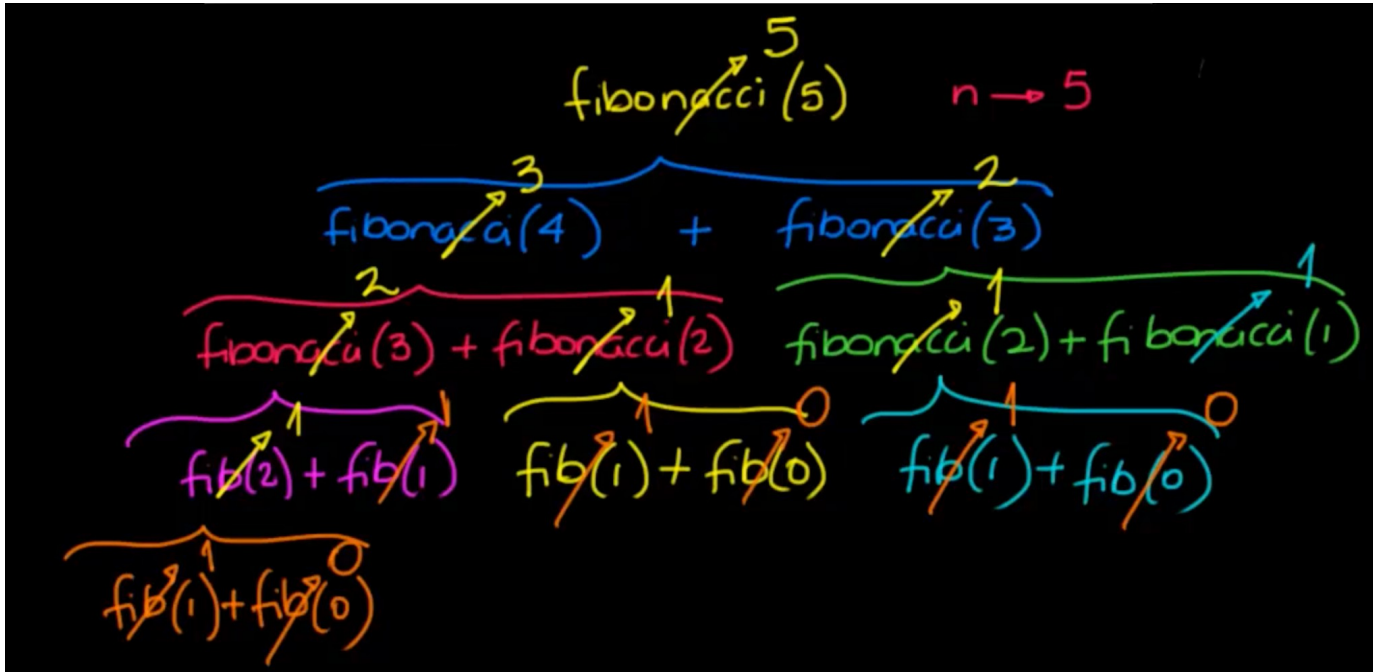
Sucesión de Fibonacci (Recursiva)

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597 ...

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2}$$



Sucesión de Fibonacci en Java (Recursiva)

```
public static int fibonacci (int n){  
    if (n==0 || n==1){ \\Caso base  
        return n;  
    }  
    else{ \\Caso recursivo  
        return fibonacci(n-1) + fibonacci(n-2);  
    }  
}
```

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2}$$

```
1  import java.util.Scanner;  
2  
3  public class Recursividad_Fibonacci {  
4  
5      public static void main (String[] args){  
6          Scanner sc = new Scanner(System.in);  
7          System.out.println("Digite un numero para calcular su serie de Fibonacci: ");  
8  
9          int num = sc.nextInt();  
10         System.out.println("El fibonacci de " + num + " es = " + fibonacci(num));  
11     }  
12  
13     public static int fibonacci (int n){  
14         if (n==0 || n==1){  
15             return n;  
16         }  
17         else{  
18             return fibonacci(n-1) + fibonacci(n-2);  
19         }  
20     }  
21 }
```

Sucesión de Fibonacci en Java (con ciclos)

```
1 public static int fibonacciCicloWhile (int n){
2     int fibon = 0, primero = 1, segundo = 0;
3
4     while (n>0){
5         fibon = primero + segundo;
6         primero = segundo;
7         segundo = fibon;
8         n = n-1;
9     }
10    return fibon;
11 }
```

While-loop*

```
1 public static int fibonacciCicloFor (int n){
2     int fibon = 0, primero = 1, segundo = 0;
3
4     for (int i=1; i<=n; i++){
5         fibon = primero + segundo;
6         primero = segundo;
7         segundo = fibon;
8     }
9
10    return fibon;
11 }
```

For-loop*

```
1 public static int fibonacci (int n){
2     if (n==0 || n==1){
3         return n;
4     }
5     else{
6         return fibonacci(n-1) + fibonacci(n-2);
7     }
8 }
```

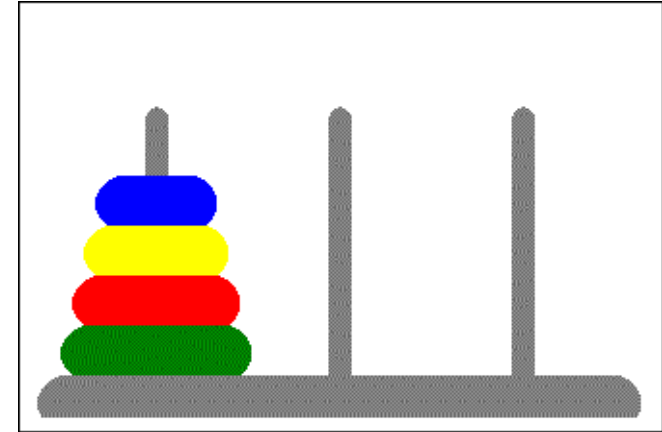
Recursivo

***Hacer prueba de escritorio si es necesaria**

4. Torres de Hanoi

Problema de las Torres de Hanoi

- Es un rompecabezas o juego matemático inventado en 1883 por el matemático francés Édouard Lucas.
- Consiste en un número de discos perforados de radio creciente que se apilan insertándose en uno de los tres postes fijados a un tablero.
- El objetivo del juego es trasladar la pila a otro de los postes siguiendo ciertas reglas:
 - Solo se puede mover un disco cada vez.
 - No se puede colocar un disco más grande encima de un disco más pequeño.
 - Solo se puede desplazar el disco que se encuentre arriba en cada poste.

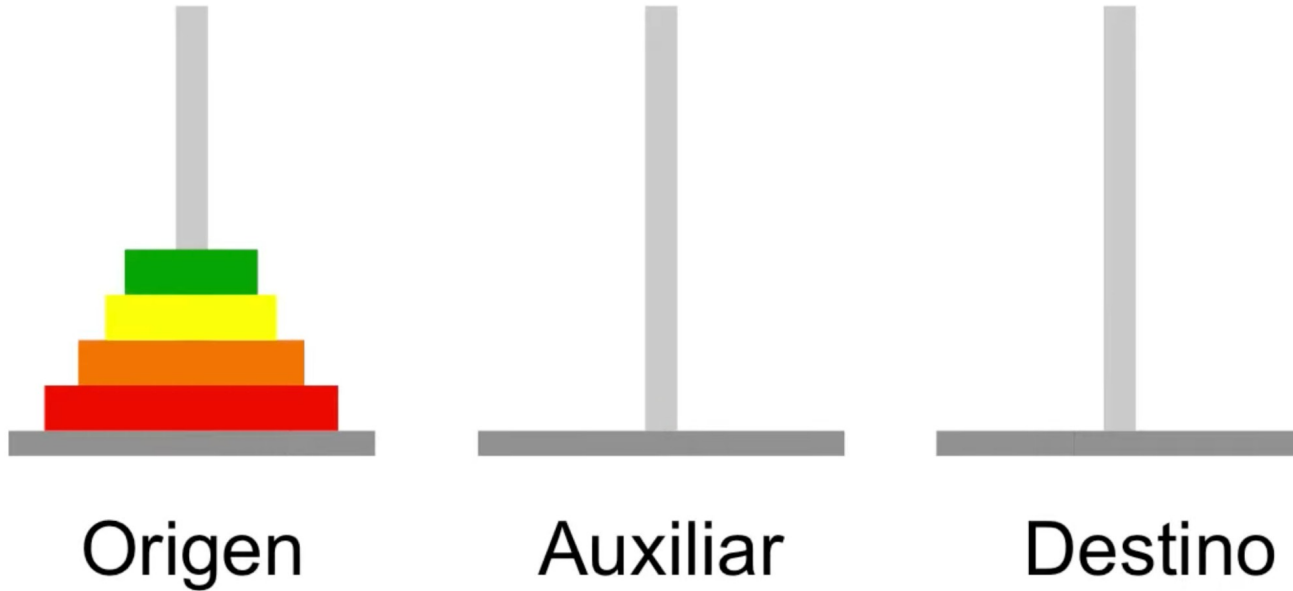


Leyenda de los monjes

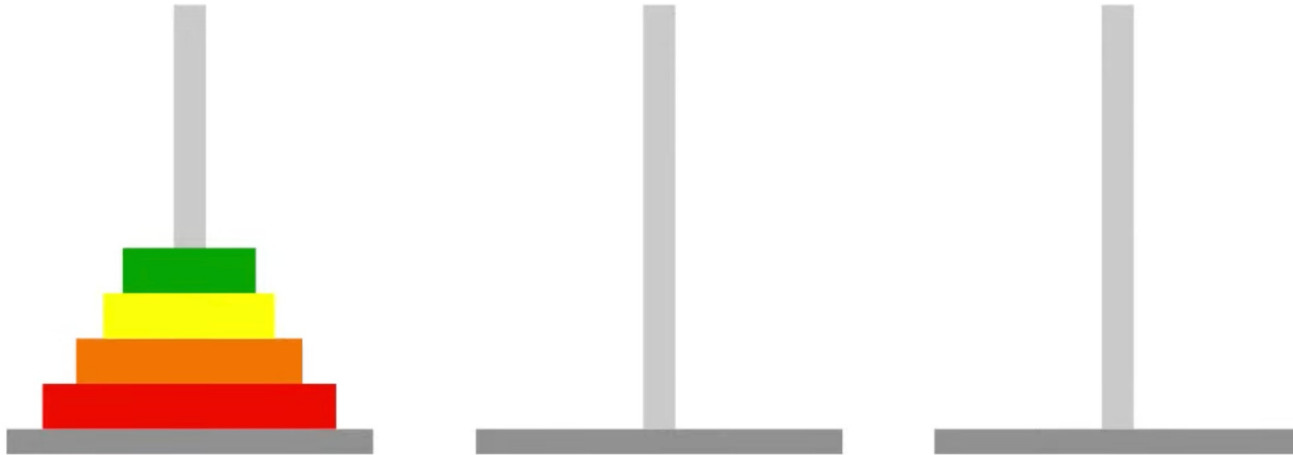
- Unos monjes en Hanói (capital de Vietnam) tienen que mover 64 discos de una torre a la otra.
- Cada disco es muy frágil, entonces uno grande no puede ir encima de uno pequeño.
- La leyenda dice, **que el mundo se acabará al completar su hazaña**



Estructura recursiva Torres de Hanoi



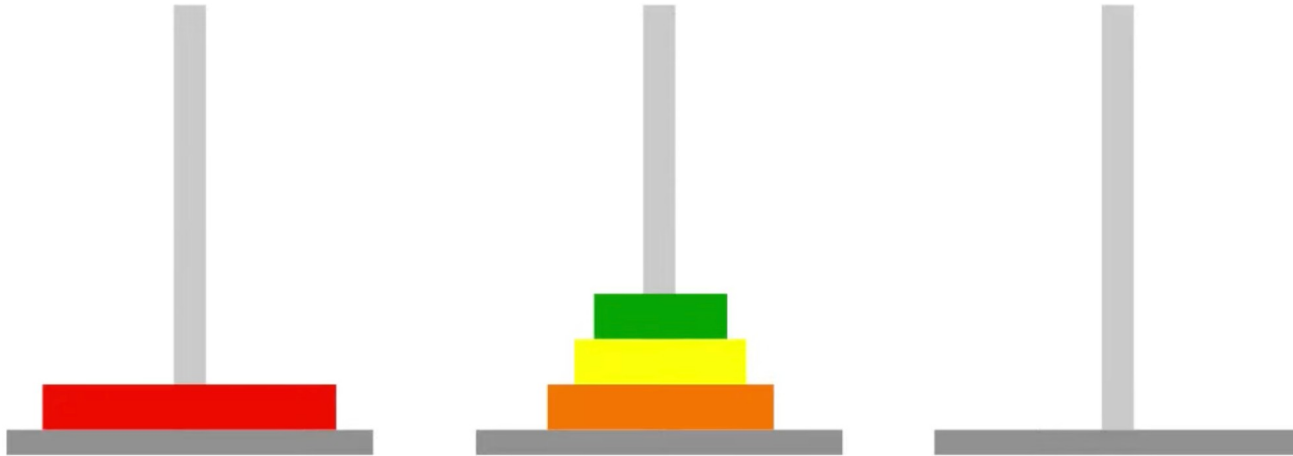
Estructura recursiva Torres de Hanoi



Pasos:

1. Para n discos, la idea es pasar los $n-1$ discos de la torre **origen** a la **auxiliar**.
2. Después, mover el disco más grande (n) de la torre **origen** a la torre **destino**.
3. Finalmente, mover los $n-1$ discos de la torre **auxiliar** a la torre **destino**.

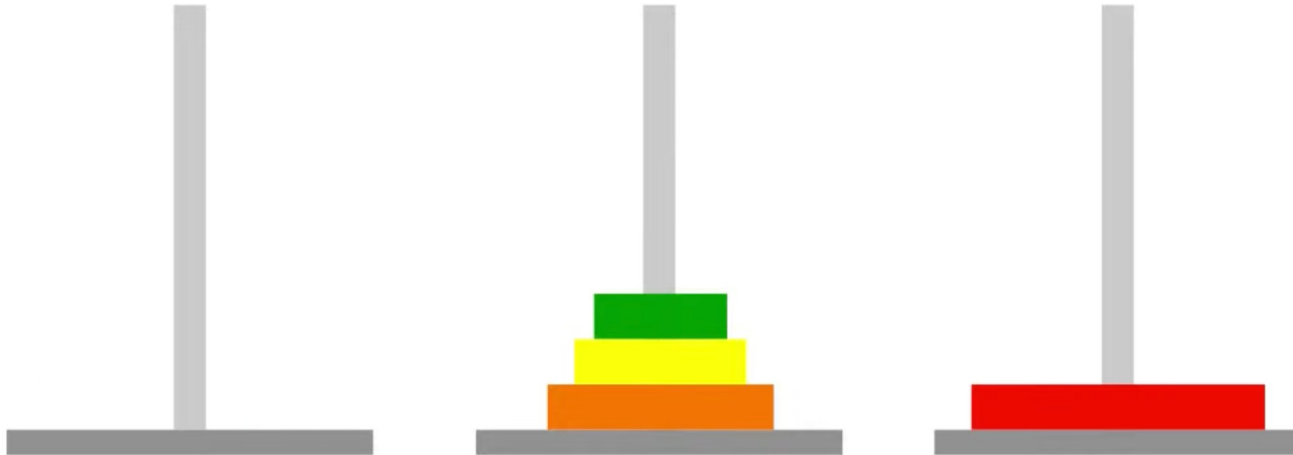
Estructura recursiva Torres de Hanoi



Paso 1:

Para n discos, la idea es pasar los $n-1$ discos de la torre **origen** a la **auxiliar**.

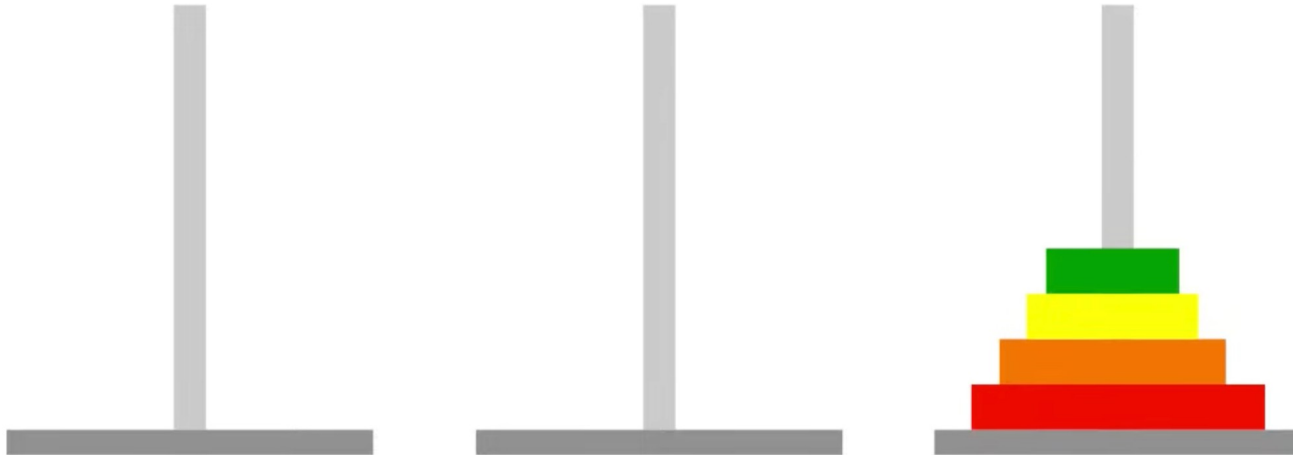
Estructura recursiva Torres de Hanoi



Paso 2.

Mover el disco más grande de la torre **origen a la torre **destino**.**

Estructura recursiva Torres de Hanoi



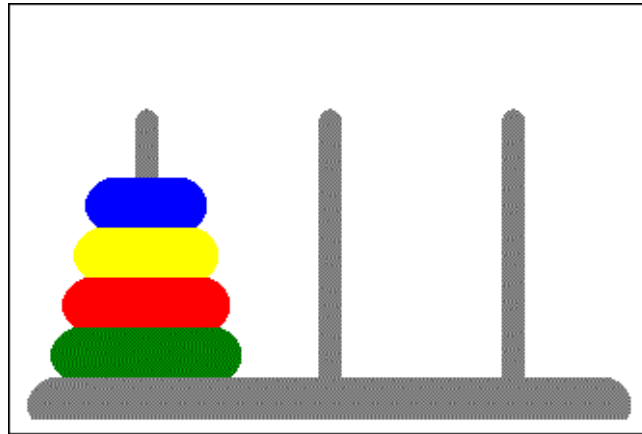
Paso 3:

Finalmente, mover los **n-1** discos de la torre **auxiliar** a la torre **destino**.

Let's play

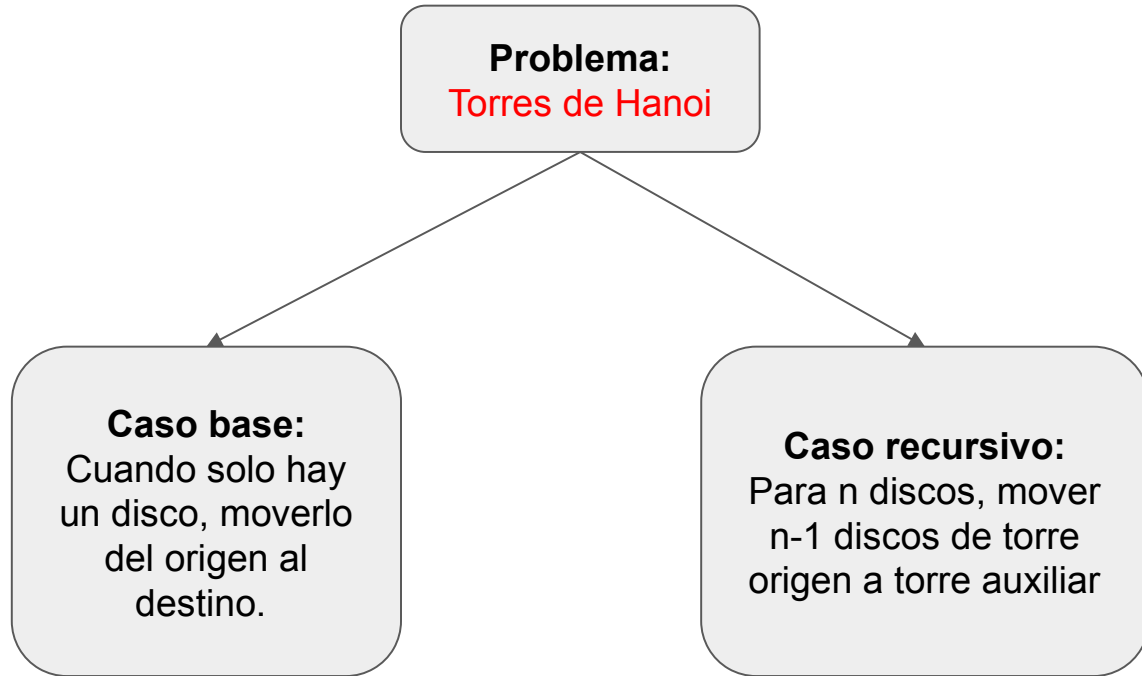
<https://www.geogebra.org/m/NqyWJVra>

<https://www.disfrutalasmaticas.com/juegos/torre-de-hanoi-2.html>



Estructura recursiva Torres de Hanoi

¿Cómo sería el pseudocódigo?



Estructura recursiva Torres de Hanoi

Pasos:

1. Para **n** discos, la idea es pasar los **n-1** discos de la torre **origen** a la **auxiliar**.
2. Después, mover el disco más grande (**n**) de la torre **origen** a la torre **destino**.
3. Finalmente, mover los **n-1** discos de la torre **auxiliar** a la torre **destino**.

¿Cómo sería el pseudocódigo?

```
function hanoi(n, origen, destino, auxiliar){  
    Paso 1: hanoi(n-1, origen, auxiliar, destino)  
    Paso 2: mover disco n de origen a destino  
    Paso 3: hanoi(n-1, auxiliar, destino, origen)  
}
```

Torres de Hanoi (Recursivo)

- Función recursiva:

```
1 function hanoi(n, origen, destino, auxiliar){  
2   if (n>0){  
3     hanoi(n-1, origen, auxiliar, destino)  
4     mover disco n de origen a destino  
5     hanoi(n-1, auxiliar, destino, origen)  
6   }  
7 }
```

- La fórmula para encontrar el número de movimientos necesarios para transferir n discos es: $2^n - 1$.

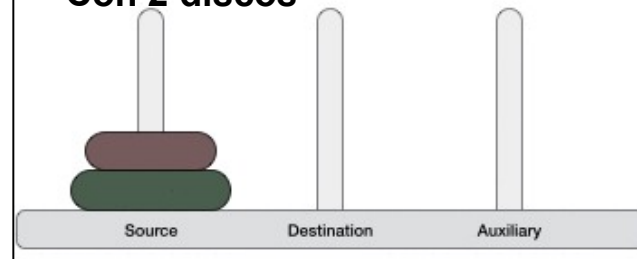
$$a_n = a_{n-1} + 1 + a_{n-1}$$

$$a_n = 2a_{n-1} + 1 \quad ; \text{ con } a_1 = 1$$

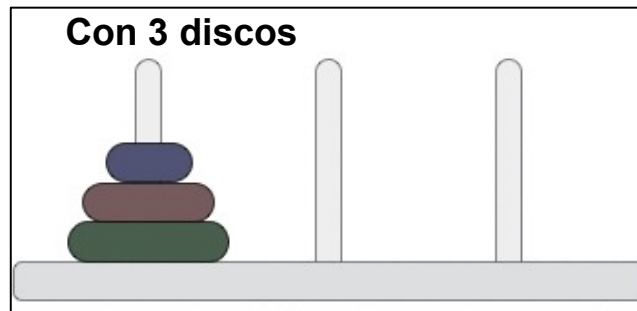
Nota #1: La sucesión sigue la forma **1, 3, 7, 15, ...** $a_n = 2^n - 1$

Nota #2: Para el caso de la leyenda $2^{64} - 1 = 1.84e+19$ movimientos (584000 millones de años, asumiendo un movimiento por segundo)

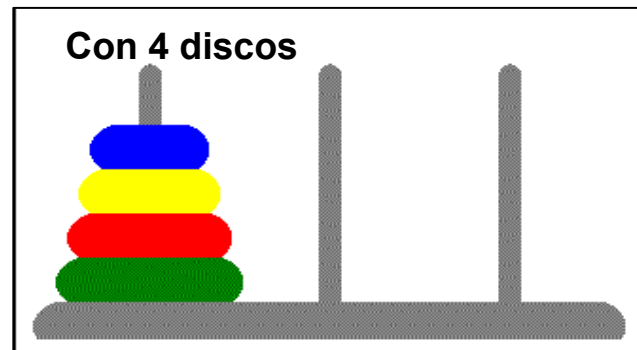
Con 2 discos



Con 3 discos



Con 4 discos



Torres de Hanoi en Java

That's just a toy, for children



CS students



```

1  import java.util.Scanner;
2
3  public class Recursividad_TorresHanoi {
4
5      public static void main(String[] args){
6          Scanner sc = new Scanner(System.in);
7          System.out.println("Digite el número de discos para la torre de Hanoi: ");
8
9          int num = sc.nextInt();
10         torresHanoi(num, 1, 2, 3);
11         System.out.println("Termina el juego!");
12     }
13
14     public static void torresHanoi(int numeroDiscos, int torreOrigen, int torreDestino, int torreAuxiliar){
15         if (numeroDiscos > 0){
16             torresHanoi(numeroDiscos-1, torreOrigen, torreAuxiliar, torreDestino);
17             System.out.println("Mover disco " + numeroDiscos + ": De torre " + torreOrigen + " a torre " + torreDestino);
18             torresHanoi(numeroDiscos-1, torreAuxiliar, torreDestino, torreOrigen);
19         }
20     }
21 }
    
```

4. Relaciones de recurrencia

Relación de Recurrencia

Ecuación que define una secuencia recursiva.

Cada término de la secuencia es definido como una función de términos anteriores.

Factorial:

$$F_n = n F_{n-1}, \quad \text{con valor inicial } F_1 = 1$$

Fibonacci:

$$F_n = F_{n-1} + F_{n-2}, \quad \text{con valores iniciales } F_0 = 0, F_1 = 1$$

Torres de Hanoi:

$$T_n = 2 T_{n-1} + 1, \quad \text{con valor inicial } T_1 = 1$$

Objetivo: Encontrar una expresión que satisfaga la relación de recurrencia pero que NO SEA RECURSIVA.

Ejemplo: $a_n = 2a_{n-1}$; con $a_1 = 4$ $\longrightarrow$ Recursiva

$$a_2 = 2a_1 = 2(4) = 8$$

$$a_3 = 2a_2 = 2(8) = 16$$

$$a_4 = 2a_3 = 2(16) = 32$$

Secuencia: $\{4, 8, 16, 32, \dots\} \rightarrow a_n = 2^{n+1} \longrightarrow$ NO Recursiva

Clasificación de relaciones de recurrencia:

ORDEN: Mayor diferencia entre los subíndices de los **elementos de la sucesión** en la relación de recurrencia. ¿Cuántos términos anteriores hay que conocer para obtener uno en particular?

GRADO: Mayor exponente al que están elevados los **elementos de la sucesión** que figuran en la relación de recurrencia.

ECUACIÓN HOMOGÉNEA: No contiene términos independientes (solo contiene recurrencias). Términos independientes son todos aquellos en los que NO figuran elementos de la sucesión. Ejemplo: $4, 3n, n^2$.

COEFICIENTES CONSTANTES: Aquellos que no dependen de n .

EJERCICIO

Relación de recurrencia	Orden	Grado	Homogénea		Coeficientes	
			SI	NO	Const	Var
$a_n = a_{n-1} + 2$						
$a_n = a_{n-1} + a_{n-2}$						
$a_n = a_{n-1} + 2n^2 - 1$						
$a_n = n a_{n-2}$						
$a_n = 2a_{n-1}$						
$a_{n+1} = 3a_{n-1}$						
$a_n^2 = 2a_{n-1} - 1$						

Clasificación de relaciones de recurrencia:

- **ORDEN:** Mayor diferencia entre los subíndices de la relación de recurrencia.
- **GRADO:** Mayor exponente al que están elevados los elementos de la sucesión.
- **ECUACIÓN HOMOGÉNEA:** No contiene términos independientes (solo contiene recurrencias).
- **COEFICIENTES CONSTANTES:** Aquellos que no dependen de n.

EJERCICIO

Relación de recurrencia	Orden	Grado	Homogénea		Coeficientes	
			SI	NO	Const	Var
$a_n = a_{n-1} + 2$	1	1		X	X	
$a_n = a_{n-1} + a_{n-2}$	2	1	X		X	
$a_n = a_{n-1} + 2n^2 - 1$	1	1		X	X	
$a_n = na_{n-2}$	2	1	X			X
$a_n = 2a_{n-1}$	1	1	X		X	
$a_{n+1} = 3a_{n-1}$	2	1	X		X	
$a_n^2 = 2a_{n-1} - 1$	1	2		X	X	

Clasificación de relaciones de recurrencia:

- **ORDEN:** Mayor diferencia entre los subíndices de la relación de recurrencia.
- **GRADO:** Mayor exponente al que están elevados los elementos de la sucesión.
- **ECUACIÓN HOMOGÉNEA:** No contiene términos independientes (solo contiene recurrencias).
- **COEFICIENTES CONSTANTES:** Aquellos que no dependen de n.

Relaciones de recurrencia lineales de orden 1, homogéneas

$$a_n = k a_{n-1}, \text{ con } k \text{ una constante real no nula}$$

————→ Recursiva

Resolución: Suponemos a_0 conocido, entonces: $a_1 = k a_0$

$$a_2 = k a_1 \quad a_2 = k k a_0$$

$$a_2 = k^2 a_0$$

$$a_3 = k a_2$$

$$a_3 = k k^2 a_0$$

$$a_3 = k^3 a_0$$

————→ NO Recursiva
→ $a_n = k^n a_0$

Demostración por principio de inducción completa:

Paso base: $n = 0:$ $a_0 = k^0 a_0 = a_0$

Paso inductivo: Hipótesis: $n = h$ $a_h = k^h a_0$

Tesis: $n = h+1$ $a_{h+1} = k^{h+1} a_0$

Demostración: $a_{h+1} = k a_h = k k^h a_0 = k^{h+1} a_0$ —————→ Queda demostrado que son equivalentes

Ejercicio:

$$3a_n - 5a_{n-1} = 0 \text{ con } a_0 = 4$$

$a_n = 5/3 a_{n-1}$ sigue la forma $a_n = k a_{n-1}$ que dijimos era equivalente a $a_n = k^n a_0$, es decir: $a_n = (5/3)^n \cdot a_0 = (5/3)^n \cdot 4$

Paso base: $n=0:$ $a_0 = (5/3)^0 \cdot a_0 = 4$

Paso inductivo: **Hipótesis:** $n=h$ $a_h = (5/3)^h \cdot 4$

Tésis: $n = h+1$ $a_{h+1} = (5/3)^{h+1} \cdot 4$

Demostración: $a_{h+1} = (5/3)a_h = (5/3)(5/3)^h \cdot 4 = (5/3)^{h+1} \cdot 4$ $\longrightarrow$ Queda demostrado que son equivalentes

Relaciones de recurrencia lineales de orden 2, homogéneas

$$c_n a_n + c_{n-1} a_{n-1} + c_{n-2} a_{n-2} = 0 \longrightarrow \text{Índices de mayor a menor}$$

Resolución: Suponemos que $a_n = x^n$ es una solución de la ecuación

$$c_n x^n + c_{n-1} x^{n-1} + c_{n-2} x^{n-2} = 0 \Rightarrow x^{n-2} (c_n x^2 + c_{n-1} x + c_{n-2}) = 0 \Rightarrow c_n x^2 + c_{n-1} x + c_{n-2} = 0 \quad \text{Ecuación característica}$$

Raíces de la ecuación característica: Reales distintas $\rightarrow$ Sin gral: $a_n = k_1 r_1^n + k_2 r_2^n$

Reales iguales $\rightarrow$

Sin gral:

$$a_n = k_1 r_1^n + k_2 r_2^n$$

Ejemplo: $a_n - 5a_{n-1} + 6a_{n-2} = 0$ con $a_0 = 3$ y $a_1 = 5$

NOTA: Al ser de orden 2, se requieren 2 datos iniciales para encontrar la solución particular de la recursión.

Ec. Característica: $x^2 - 5x + 6 = 0$

Raíces: $r_1 = 2$ y $r_2 = 3$

Solución Gral: $a_n = k_1 \cdot 2^n + k_2 \cdot 3^n$

Solución particular: $a_0 = 3 \rightarrow 3 = k_1 \cdot 2^0 + k_2 \cdot 3^0 \rightarrow k_1 + k_2 = 3$

$$k_1 = 3 - k_2$$

$$2(3 - k_2) + 3k_2 = 5 \rightarrow 6 - 2k_2 + 3k_2 = 5 \rightarrow 6 + k_2 = 5$$

$$k_2 = -1 \text{ y } k_1 = 4$$

$$a_1 = 5 \rightarrow 5 = k_1 \cdot 2^1 + k_2 \cdot 3^1 \rightarrow 2k_1 + 3k_2 = 5 \rightarrow$$

Sin

Ejercicio en clase: $a_{n+2}=4a_{n+1}-4a_n$ con $a_0=1$ y $a_1=3$

Tarea próxima clase:

Encontrar SIn particular para:

- Fibonacci
- Torres de Hanoi